

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE CODE: CSA1401

COURSE NAME: COMPILER DESIGN

COURSE OUTCOME COVERED

CO3: *Analyze syntax-directed translation method using synthesized and inherited attributes to generate a Parser Tree. (BL4)*

Assessment Tool Weightage: 40%

QUESTIONS:5

Duration :60 Mins
On Class
Total Marks:50

Annexure A

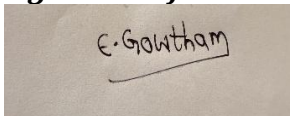
EXPERIMENTS

Code of Conduct:

I Enamala Gowtham/192311190 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

A rectangular box containing a handwritten signature in black ink. The signature appears to be 'E. Gowtham' written in a cursive style.

Annexure A

EXPERIMENTS

Q1. A calculator compiler evaluates arithmetic expressions during parsing using YACC.

- (i) Design a YACC grammar for arithmetic operators + and *.
- (ii) Embed semantic actions in the grammar rules to compute expression values.
- (iii) Evaluate the input expression:

$3 + 4 * 5$

- (iv) Display the computed result after parsing. (10 Marks)

Q2. A compiler constructs syntax trees for subsequent semantic analysis and code generation using YACC.

- (i) Define a YACC grammar for arithmetic expressions.
- (ii) Implement semantic actions to construct a parse tree / Abstract Syntax Tree (AST).
- (iii) Generate and illustrate the tree for:

$a + b * c$

- (iv) Traverse the generated tree and display the traversal order. (10 Marks)

Q3. A compiler evaluates arithmetic expressions using synthesized attributes only in a YACC-based syntax-directed translation scheme.

- (i) Implement a Syntax-Directed Definition (SDD) using YACC semantic values (\$\$, \$1, \$2, etc.).
- (ii) Evaluate the expression:

$2 * 3 + 4$

- (iii) Show how attribute values are propagated during reductions.
- (iv) Demonstrate the bottom-up evaluation process performed by the parser. (10 Marks)

Q4. A compiler developed using YACC must verify type compatibility between variables and user-defined types. The system distinguishes between name equivalence and structural equivalence during semantic analysis.

- (i) Design a YACC grammar to process type declarations and variable declarations.
- (ii) Implement semantic actions and symbol table routines to compare type expressions.
- (iii) Differentiate and verify name equivalence and structural equivalence for the following declarations:

```
type A = int;
```

```
type B = int;
```

```
A x;
```

```
B y;
```

- (iv) Test the parser with multiple type declarations and assignment statements.
- (v) Display the type compatibility result and indicate whether the types are equivalent under:

- (a). Name Equivalence

- (b). Structural Equivalence (10 Marks)

Q5. A compiler implemented using YACC processes assignment statements involving multiple data types (int, float). During semantic analysis, the compiler performs automatic type conversion (coercion).

- (i) Design a YACC grammar for variable declarations and assignment statements.
- (ii) Analyze the following program fragment:

```
float temperature;
```

```
int sensor_value;
```

```
temperature = sensor_value;
```

- (iii) Implement semantic actions and symbol table entries to support type promotion rules.

(iv) Demonstrate how the compiler performs implicit type conversion (int → float) during assignment.

(v) Display the resulting type information and the final type of the variable after assignment. (10 Marks)

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25	Demonstrates thorough understanding and effectively integrates multiple concepts/technologies	Good understanding with proper integration of most concepts	Basic understanding; limited or partial integration	Poor understanding; unable to integrate concepts
Experimental Design	30	Well-planned, systematic implementation with optimal solution	Correct implementation with minor issues	Basic implementation with errors or inefficiencies	Incorrect or incomplete implementation
Use of Tools	20	Effective and advanced use of tools/technologies with proper configuration	Appropriate use of tools with correct execution	Limited or inconsistent use of tools	Incorrect or minimal use of tools
Analysis of Results	15	Insightful analysis with accurate interpretation and validation of results	Good analysis with reasonable interpretation	Basic analysis with limited insights	Poor or incorrect analysis of results
Documentation	10	Clear, well-structured documentation with diagrams, code, and results	Organized documentation with necessary details	Basic documentation with missing details	Poor or incomplete documentation

Q1. A Calculator Compiler Evaluates Arithmetic Expressions Using YACC

Aim: To design a YACC grammar for arithmetic expressions containing + and *, embed semantic actions to calculate expression values, evaluate $3 + 4 * 5$, and display the result. The assessment specifically requires grammar design, semantic actions, evaluation, and displaying the computed result.

1. Introduction

YACC (Yet Another Compiler-Compiler) is a parser generator used to create parsers from a context-free grammar.

In this experiment, YACC is used not only to recognize an arithmetic expression but also to calculate its value during parsing.

The grammar contains:

- E – Expression
- T – Term
- F – Factor
- NUMBER – Numeric value

The grammar is designed so that multiplication has higher precedence than addition.

2. Grammar

The grammar is:

$E \rightarrow E + T$

$E \rightarrow T$

$T \rightarrow T * F$

$T \rightarrow F$

$F \rightarrow \text{NUMBER}$

Meaning

E represents an entire expression.

T represents a term containing multiplication.

F represents a number.

The grammar naturally gives * higher precedence because multiplication occurs inside T, while addition occurs at the E level.

3. Semantic Actions

YACC uses special symbols to access semantic values.

- \$1 = semantic value of first RHS symbol
- \$2 = semantic value of second RHS symbol
- \$3 = semantic value of third RHS symbol
- \$\$ = semantic value of the LHS symbol

Semantic rules:

$E \rightarrow E + T \quad \{ \$\$ = \$1 + \$3; \}$

$E \rightarrow T \quad \{ \$\$ = \$1; \}$

$T \rightarrow T * F \quad \{ \$\$ = \$1 * \$3; \}$

$T \rightarrow F \quad \{ \$\$ = \$1; \}$

$F \rightarrow \text{NUMBER} \quad \{ \$\$ = \$1; \}$

4. Evaluation of $3 + 4 * 5$

Input: $3 + 4 * 5$

First, multiplication is evaluated: $4 * 5 = 20$

Then addition is evaluated: $3 + 20 = 23$

Therefore: $3 + 4 * 5 = 23$

5. Reduction Process

The parser can conceptually reduce the expression as:

$3 \rightarrow F \rightarrow T$

$4 \rightarrow F$

$5 \rightarrow F$

$4 * 5 \rightarrow T$

$3 + 20 \rightarrow E$

The final semantic value of E is:

$E.val = 23$

6. Expected Output

Enter expression: $3 + 4 * 5$

Result = 23

7. Analysis

The important point is that the parser does not simply recognize the expression.

The semantic actions calculate its value as reductions occur.

The grammar also prevents the expression from being interpreted as: $(3 + 4) * 5$

Instead, it correctly interprets it as: $3 + (4 * 5)$

Result:

Thus, the YACC-based calculator successfully parses and evaluates the expression: $3 + 4 * 5$ and produces: 23

Q2. YACC Construction of Syntax Tree / Abstract Syntax Tree

Aim

To define a YACC grammar for arithmetic expressions, construct an Abstract Syntax Tree using semantic actions, generate the tree for $a + b * c$, and display traversal orders.

This question specifically requires construction of the parse tree/AST and traversal of the generated tree.

1. Introduction

A syntax tree represents the structure of an expression.

An **Abstract Syntax Tree (AST)** removes unnecessary grammar details and keeps the important operators and operands.

For example:

$a + b * c$

is represented using:

- $+$ as the root operator
- a as the left operand
- $*$ as the right subtree
- b and c as operands of $*$

2. Grammar

$E \rightarrow E + T$

$E \rightarrow T$

$T \rightarrow T * F$

$T \rightarrow F$

$F \rightarrow ID$

Here:

- E = expression
- T = term
- F = factor
- ID = identifier

3. AST Construction

For every operator, create a node.

For example:

$E \rightarrow E + T$

creates a $+$ node.

Similarly:

$T \rightarrow T * F$

creates a * node.

For:

$F \rightarrow ID$

an identifier node is created.

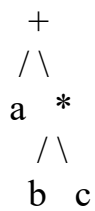
4. Expression

$a + b * c$

Since multiplication has higher precedence:

$a + (b * c)$

5. AST Diagram



The root is +.

The left child is a.

The right child is *.

The children of * are b and c.

6. Traversals

Preorder

Preorder follows:

Root $\rightarrow$ Left $\rightarrow$ Right

Therefore:

$+ a * b c$

Inorder

Inorder follows:

Left $\rightarrow$ Root $\rightarrow$ Right

Therefore:

$a + b * c$

Postorder

Postorder follows:

Left $\rightarrow$ Right $\rightarrow$ Root

Therefore:

$a b c * +$

7. AST Interpretation

The postorder representation:

a b c * +

can be interpreted as:

b c *

which gives:

b * c

and then:

a (b*c) +

which gives:

a + (b*c)

This demonstrates that the AST correctly represents operator precedence.

8. Analysis

ASTs are useful because later compiler phases can work directly with the tree.

For example, the compiler can use the AST for:

- semantic analysis
- type checking
- intermediate code generation
- optimization
- final code generation

Result

The AST for:

a + b * c

is successfully constructed as:

```
      +
     /\
    a  *
     /\
    b  c
```

The traversal results are:

Preorder : + a * b c

Inorder : a + b * c

Postorder : a b c * +

Q3. Syntax-Directed Translation Using Synthesized Attributes

Aim: To implement a Syntax-Directed Definition using YACC semantic values such as \$\$, \$1, and \$2, evaluate $2 * 3 + 4$, show attribute propagation, and demonstrate bottom-up evaluation.

1. Introduction

A **Syntax-Directed Definition (SDD)** associates semantic rules with grammar productions.

A synthesized attribute is calculated from the attributes of child nodes.

For example:

$E \rightarrow E + T$

can have:

$E.val = E1.val + T.val$

The value is passed upward from the children to the parent.

2. Grammar

$E \rightarrow E + T$

$E \rightarrow T$

$T \rightarrow T * F$

$T \rightarrow F$

$F \rightarrow \text{NUMBER}$

3. Semantic Rules

$E \rightarrow E + T \quad \{ E.val = E1.val + T.val \}$

$E \rightarrow T \quad \{ E.val = T.val \}$

$T \rightarrow T * F \quad \{ T.val = T1.val * F.val \}$

$T \rightarrow F \quad \{ T.val = F.val \}$

$F \rightarrow \text{NUMBER} \quad \{ F.val = \text{NUMBER.lexval} \}$

In YACC form:

$E : E '+' T \{ \$\$ = \$1 + \$3; \}$

$E : T \quad \{ \$\$ = \$1; \}$

$T : T '*' F \{ \$\$ = \$1 * \$3; \}$

$T : F \quad \{ \$\$ = \$1; \}$

F : NUMBER { \$\$ = \$1; }

4. Meaning of YACC Symbols

Symbol Meaning

\$1 Semantic value of first symbol
\$2 Semantic value of second symbol
\$3 Semantic value of third symbol
\$\$ Semantic value of LHS
\$0 Semantic value before the current production

For this experiment, \$1, \$3, and \$\$ are mainly used.

5. Input

$2 * 3 + 4$

6. Evaluation

First:

$2 * 3 = 6$

Then:

$6 + 4 = 10$

Therefore:

Result = 10

7. Attribute Propagation

The values move upward during reduction.

First:

NUMBER = 2

F.val = 2

T.val = 2

Next:

NUMBER = 3

F.val = 3

Now:

$T \rightarrow T * F$

Therefore:

T.val = $2 * 3$

T.val = 6

Next:

NUMBER = 4

F.val = 4

T.val = 4

Finally:

$E \rightarrow E + T$

Therefore:

$E.val = 6 + 4$

$E.val = 10$

8. Reduction Table

S.no	Step Reduction	Result
1	$F \rightarrow \text{NUMBER}(2)$	2
2	$T \rightarrow F$	2
3	$F \rightarrow \text{NUMBER}(3)$	3
4	$T \rightarrow T * F$	6
5	$F \rightarrow \text{NUMBER}(4)$	4
6	$T \rightarrow F$	4
7	$E \rightarrow E + T$	10

9. Bottom-Up Evaluation

YACC uses a bottom-up parsing approach.

The parser first recognizes the smaller elements:

2

3

4

Then reduces them:

$2 \rightarrow F \rightarrow T$

$3 \rightarrow F$

$T * F \rightarrow T$

$4 \rightarrow F \rightarrow T$

$E + T \rightarrow E$

The final result is stored in the synthesized attribute: $E.val = 10$

10. Analysis

The important feature of synthesized attributes is that information flows from children to parent.

Therefore:

NUMBER

↓

F

↓

T

↓

E

This is suitable for bottom-up parsing because YACC performs reductions from smaller grammar components to larger ones.

Result

The expression: $2 * 3 + 4$ is successfully evaluated using synthesized attributes.

Final result: 10

Q4. Name Equivalence and Structural Equivalence

Aim

To design a YACC grammar for type declarations and variable declarations, implement semantic actions and symbol-table routines, and verify name and structural equivalence.

The assessment specifically requires testing:

type A = int;

type B = int;

A x;

B y; and displaying compatibility under both equivalence rules.

1. Introduction

A compiler needs to determine whether two types are compatible.

Two important methods are:

1. Name equivalence
2. Structural equivalence
- 3.

2. Type Declarations

Given:

type A = int;

type B = int;

Both A and B represent int, but they are separately declared type names.

Variable declarations:

A x;

B y;

Therefore:

$x \rightarrow A$

$y \rightarrow B$

3. Symbol Table

A symbol table can contain:

Identifier Type

A	int
B	int
x	A
y	B

The compiler uses this table during semantic analysis.

4. Name Equivalence

Name equivalence considers two types equivalent when they have the same declared type identity.

Here:

A

and

B

are two different declared type names.

Therefore:

$A \neq B$

Hence:

$x = y$

would not be accepted under strict name equivalence because:

$\text{type}(x) = A$

$\text{type}(y) = B$

and:

$A \neq B$

Name Equivalence Result

$A \text{ and } B \rightarrow \text{NOT EQUIVALENT}$

5. Structural Equivalence

Structural equivalence compares the actual structure of the types.

Here:

$A \rightarrow \text{int}$

$B \rightarrow \text{int}$

Therefore:

$\text{structure}(A) = \text{structure}(B)$

Hence:

$A \equiv B$

Structural Equivalence Result

$A \text{ and } B \rightarrow \text{EQUIVALENT}$

6. Comparison Table

Property	Name Equivalence	Structural Equivalence
Comparison basis	Type name/identity	Type structure
A	int	int
B	int	int
A vs B	Not equivalent	Equivalent
A x vs B y	Incompatible	Compatible

7. Assignment Test

Consider:

A x;

B y;

$x = y;$

Name Equivalence

$A \neq B$

Therefore:

Assignment rejected

Structural Equivalence

$A \rightarrow \text{int}$

$B \rightarrow \text{int}$

Therefore:

Assignment accepted

8. Multiple Declaration Test

Another example:

`type C = float;`

`C z;`

Symbol table:

$C \rightarrow \text{float}$

$z \rightarrow C$

Comparing A and C:

$A \rightarrow \text{int}$

$C \rightarrow \text{float}$

Therefore:

$A \neq C$

under both name and structural equivalence.

9. Analysis

This experiment shows why a compiler needs type information during semantic analysis.

The symbol table allows the compiler to determine:

- declared type
- type identity
- underlying type
- assignment compatibility

Result

For:

`type A = int;`

`type B = int;`

the result is:

Name Equivalence $\rightarrow$ NOT EQUIVALENT

Structural Equivalence $\rightarrow$ EQUIVALENT

Q5. Automatic Type Conversion / Type Coercion

Aim

To design a YACC grammar for variable declarations and assignments, analyze the given program, implement type promotion rules, and demonstrate implicit conversion from int to float.

The assessment requires specifically demonstrating $\text{int} \rightarrow \text{float}$ conversion.

1. Given Program

```
float temperature;  
int sensor_value;  
temperature = sensor_value;
```

2. Introduction

Different variables may have different data types.

Here:

$\text{temperature} \rightarrow \text{float}$

$\text{sensor_value} \rightarrow \text{int}$

The assignment is:

$\text{temperature} = \text{sensor_value};$

The compiler must check whether the types are compatible.

3. Symbol Table

After processing declarations:

Variable	Type
----------	------

temperature	float
-------------	-------

sensor_value	int
--------------	-----

4. Type Promotion

The compiler has the promotion rule:

$\text{int} \rightarrow \text{float}$

This is called **implicit type conversion** or **coercion**.

The compiler automatically converts the integer value into a floating-point value before assignment.

5. Assignment Analysis

Original statement:

$\text{temperature} = \text{sensor_value};$

Types:

LHS = float

RHS = int

Compiler checks:

int $\rightarrow$ float

The conversion is allowed.

Conceptually, it becomes:

temperature = (float)sensor_value;

6. Example

Suppose:

sensor_value = 25;

The compiler converts:

25

into:

25.0

Therefore:

temperature = 25.0;

7. Semantic Action Logic

The semantic analyzer can follow this logic:

If LHS type = float

and RHS type = int

Then: perform int $\rightarrow$ float conversion accept assignment

Otherwise, if the types are incompatible: Type mismatch

8. Type Checking Table

LHS RHS Conversion Result

float	int	int $\rightarrow$ float	Valid
-------	-----	-------------------------	-------

int	int	None	Valid
-----	-----	------	-------

float	float	None	Valid
-------	-------	------	-------

int	float	float $\rightarrow$ int	Requires narrowing/conversion handling
-----	-------	-------------------------	--

For the given experiment, the required case is:

float $\leftarrow$ int

so the compiler performs:

int $\rightarrow$ float

9. Expected Output

Type Coercion Analysis

Left variable : temperature (float)

Right variable : sensor_value (int)

Implicit conversion: int $\rightarrow$ float

Assignment is valid.

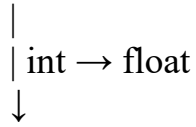
Final type of temperature = float

10. Analysis

Type coercion allows a compiler to automatically convert one compatible type into another.

In this experiment:

sensor_value



temperature

The conversion happens during semantic analysis.

The symbol table provides the type information required to make this decision.

Result

The compiler successfully identifies:

temperature $\rightarrow$ float

sensor_value $\rightarrow$ int

and performs: int $\rightarrow$ float

Therefore:

temperature = sensor_value;

is accepted with implicit type conversion.